

PHP: PHP vs Java o Java vs PHP

1. Declaración de Variables

- **Java:** Es fuertemente tipado, lo que significa que debes declarar el tipo de dato de una variable al momento de su creación.

```
int numero = 10;
String texto = "Hola, mundo";
```

PHP: Es débilmente tipado, lo que permite declarar variables sin especificar su tipo. PHP infiere el tipo de dato automáticamente. Más adelante veremos que a partir de la versión 8.0 de php se puede obligar que sea fuertemente tipado, lo que evita muchos errores.

```
$numero = 10;
$texto = "Hola, mundo";
```

Ámbito de las Variables en PHP:

- **Variables locales:** son las variables declaradas dentro de una función, solo existen dentro de esa función. Es decir, no pueden ser accedidas ni modificadas fuera de ella.

```
function miFuncion() {
    $mensaje = "Hola Mundo!";
    echo $mensaje;
}
miFuncion();
// Imprime "Hola Mundo!"

echo $mensaje;
// Error: $mensaje no está definida fuera de la función
```

Variables globales:

- Las variables globales se definen fuera de cualquier función y pueden ser accedidas desde cualquier lugar en el script, excepto dentro de funciones o métodos, a menos que se utilice la palabra clave `global`.

```

$mensajeGlobal = "Hola desde fuera!";
function mostrarMensaje() {
    global $mensajeGlobal;
    // Permite el acceso a la variable global
    echo $mensajeGlobal;
}

mostrarMensaje(); // Imprime "Hola desde fuera!"

```

- Las variables globales se deben usar con precaución porque pueden hacer que el código sea más difícil de mantener y entender, ya que pueden ser modificadas desde cualquier parte del programa.

Uso de `$GLOBALS` :

PHP tiene una variable superglobal llamada `$GLOBALS` , que es un array que almacena todas las variables globales, permitiendo acceder a ellas desde cualquier parte del código, incluso dentro de funciones, sin necesidad de declararlas como `global` .

```

$mensaje = "Hola!";

function mostrarMensajeConGLOBALS() {
    echo $GLOBALS['mensaje'];
    // Acceso a la variable global usando $GLOBALS
}

mostrarMensajeConGLOBALS();
// Imprime "Hola!"

```

ES importante minimizar el uso de variables globales y preferir pasar variables como argumentos a las funciones. Esto promueve un código más organizado y menos propenso a errores. #

Constantes:

¿Cómo se declara una constante en Java?

Se usa **final** y normalmente también **static* si es una constante de clase:

```
public class Ejemplo {  
    public static final int MAX_EDAD = 100;  
}  
-----  
  
System.out.println(Config.MENSAJE_BIENVENIDA);
```

¿Cómo se declara una constante en Java

Opción 1: con `define()` (procedimental)

```
define("MAX_EDAD", 100);
```

Opción 2: con `const` (desde PHP 5.3+, solo en clases o ámbito global)

```
const MAX_EDAD = 100;
```

Acceso a constantes

	Java	PHP
Dentro de clase	<code>Clase.CONSTANTE</code>	<code>Clase::CONSTANTE</code>
Global	No constantes globales (todo dentro de clases)	Si usas <code>define</code> , puedes usarla en cualquier sitio
Ejemplo acceso	<code>Ejemplo.MAX_EDAD</code>	<code>echo MAX_EDAD;</code> o <code>echo Clase::MAX_EDAD;</code>

-

Comportamiento y reglas

Característica	Java	PHP
Inmutabilidad	Obligatorio con <code>final</code>	Obligatorio (una constante no se redefine)
Tipado	Fuertemente tipado	Débilmente tipado
Necesita inicialización	Sí, al declararse	Sí, al declararse
Ámbito de uso	Dentro de clases normalmente	Global (<code>define</code>) o clase (<code>const</code>)
Se puede cambiar el valor	No	No
Visibilidad (<code>public</code> , etc.)	Sí	Solo en clases (<code>public const ...</code>)

2. Estructura de Clases

- **Java:** Todo el código debe estar dentro de una clase, ya que es un lenguaje puramente orientado a objetos. También exige que el nombre del archivo coincida con el de la clase pública principal.

```
public class MiClase {
    public static void main(String[] args) {
        System.out.println("Hola, mundo");
    }
}
```

- **PHP:** Aunque soporta programación orientada a objetos, PHP no obliga a usar clases para todo. El código puede estar directamente en el script o dentro de clases. El uso de php con orientación a objetos, lo veremos más adelante, en un primer momento veremos los inicios de php.

```
// Código PHP sin clases
echo "Hola, mundo";

// Con clases
class MiClase {
    public function saludar() {
        echo "Hola, mundo";
    }
}
```

3. Manejo de Funciones

- **Java:** Las funciones (métodos) deben estar dentro de una clase, y sus tipos de retorno y parámetros deben ser declarados explícitamente.

```
public int sumar(int a, int b)
{
    return a + b;
}
```

- **PHP:** Las funciones pueden estar dentro o fuera de una clase y no es obligatoria declarar el tipo de retorno o de los parámetros.

```
function sumar($a, $b)
{
    return $a + $b;
}
```

4. Estructuras de Control (if, loops)

Las estructuras de control son muy similares en ambos lenguajes, pero con ligeras diferencias en la sintaxis.

Condicional if

- **Java:**

```
if (a > b) {  
    System.out.println("A es mayor que B");  
} else {  
    System.out.println("B es mayor o igual que A");  
}
```

- **PHP:**

```
if ($a > $b) {  
    echo "A es mayor que B";  
} else {  
    echo "B es mayor o igual que A";  
}  
  
if ($a > $b) {  
    echo "A es mayor que B";  
} else if ($a < $b) {  
    echo "B es mayor o igual que A";  
} else {  
    echo "Son iguales";  
}
```

Condicional switch

1. Estructura general similar:

Ambos usan la palabra clave `switch` , seguidos de paréntesis con la expresión a evaluar y bloques `case` + `break` .

2. Uso de `break` para evitar el *fall-through*:

Si no se incluye `break` , la ejecución continúa hacia el siguiente `case` .

3. Uso de `default` :

Se usa para manejar cualquier valor que no coincida con ningún `case` .

DIFERENCIAS CLAVE

Aspecto	Java	PHP
Tipos permitidos	<code>int</code> , <code>char</code> , <code>String</code> (desde Java 7), <code>enum</code>	Cualquier tipo escalar: <code>int</code> , <code>string</code> , incluso <code>bool</code> y <code>null</code>
Comparación	Comparación estricta (<code>==</code> , sin coerción)	Comparación flexible (<code>==</code> con coerción de tipos)
Fall-through por defecto	Sí, requiere <code>break</code> para evitarlo	Sí, igual que en Java
Agrupación de casos	Se pueden agrupar varios <code>case</code> sin <code>break</code>	Igual que en Java
Versión moderna	Desde Java 14: <code>switch</code> como expresión con <code>yield</code> o <code>-></code>	PHP 8.0 introdujo <code>match</code> , que es más seguro y expresivo

- Cuando dos valores de **tipos distintos** se comparan con el operador `==` , **PHP intenta convertir uno o ambos al mismo tipo** para poder compararlos. Esto es lo que se llama **coerción** (o **conversión implícita de tipos**).
- El **fall-through** ocurre cuando, al coincidir un `case` , **la ejecución continúa en el siguiente `case` sin detenerse, a menos que se use una instrucción `break`** . Ni el nuevo `switch` de Java 14+ ni el `match` de PHP 8.0 permiten fall-through

Java (versión clásica)

```
int day = 2;
switch (day) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    default:
        System.out.println("Otro día");
}
```

PHP (versión clásica)

```
$day = 2;
switch ($day) {
    case 1:
        echo "Lunes";
        break;
    case 2:
        echo "Martes";
        break;
    default:
        echo "Otro día";
}
```

VERSIONES MODERNAS

Java `switch` como expresión (desde Java 14)

```
String result = switch (day) {
    case 1 -> "Lunes";
    case 2 -> "Martes";
    default -> "Otro día";
};
```

PHP `match` (desde PHP 8.0)

```
$result = match ($day) {
    1 => "Lunes",
    2 => "Martes",
    default => "Otro día"
};
```

Tanto `switch` moderno en Java como `match` en PHP:

- No permiten *fall-through*.
- Son expresiones (devuelven un valor).
- Obligan a cubrir todos los casos o usar `default` .

DETALLE IMPORTANTE: COMPARACIÓN DE TIPOS EN PHP

```
switch ("1") {  
    case 1: echo "Coincide con 1"; // Esto se ejecuta  
    break; }
```

Esto sucede porque PHP usa `==` (coerción de tipos) , no `===` (sin coerción de tipos) lo que puede producir resultados no intuitivos si no se tiene cuidado con los tipos.

Bucles

for

Se usa cuando sabes cuántas veces quieres repetir un bloque de código.

```
for ($i = 0; $i < 10; $i++) {  
    echo $i . "\n";  
}
```

while

Repite el código mientras la condición sea verdadera.

```
$i = 0;  
while ($i < 10) {  
    echo $i . "\n";  
    $i++;  
}
```

do...while

Ejecuta al menos una vez y luego repite mientras la condición sea verdadera.

```
$i = 0;  
do {  
    echo $i . "\n";  
    $i++;  
} while ($i < 10);
```

foreach

Para recorrer arrays o colecciones (muy común en PHP).

```
$array = ['a', 'b', 'c'];  
foreach ($array as $valor) {  
    echo $valor . "\n";  
}
```

5. Comentarios

- **Java y PHP** tienen una sintaxis de comentarios muy similar:

java

```
// Comentario de una línea  
/* Comentario de múltiples líneas */
```

php

```
// Comentario de una línea  
/* Comentario de múltiples líneas */
```

6. Manejo de Excepciones

- **Java:** El manejo de excepciones es obligatorio para muchos tipos de errores. Las excepciones deben ser capturadas y gestionadas explícitamente usando `try-catch`.

```
try {  
    int resultado = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Error: " + e.getMessage());  
}
```

- **PHP:** El manejo de excepciones es opcional y se puede realizar con `try-catch`, pero no es obligatorio en muchos casos.

```
try {  
    $resultado = 10 / 0;  
} catch (Exception $e) {  
    echo "Error: " . $e->getMessage();  
}
```

7. Finalización de Líneas

- **Java:** Cada declaración debe terminar con un punto y coma (;).

```
int a = 10;
```

- **PHP:** También requiere punto y coma al final de cada declaración.

```
$a = 10;
```

8. Entrada y Salida de Datos

- **Java:** Utiliza la clase `System` para la salida y `Scanner` para la entrada.

```
import java.util.Scanner;

public class MiClase {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Escribe tu nombre:");
        String nombre = sc.nextLine();
        System.out.println("Hola " + nombre);
    }
}
```

- **PHP:** Para la salida usa `echo` o `print` , mientras que la entrada en la terminal se hace a través de formularios en aplicaciones web. La diferencia mayor entre echo y print es que echo acepta varios parámetros y print únicamente acepta uno (además echo no devuelve valor y print devuelve 0 o 1).

```

echo "Hola, mundo";
print "Hola, mundo";
print ("Hola, mundo"); // Esto indicaría que los paréntesis no son necesarios,
pero no produce error
echo ("Hola, mundo"); // Esto indicaría que los paréntesis no son necesarios,
pero no produce error

// Con dos o más argumentos
echo "Hola", "mundo";
print "Hola", "mundo"; // Error, print solo permite un argumento
print ("Hola", "mundo"); // Error, print solo permite un argumento
echo ("Hola", "mundo"); // Error, si ponemos paréntesis, solo poderemos poner un
parámetro

```

También podemos mostrar los valores de las variables

```

$nombre = "Inma";
echo "Hola, $nombre";           // Hola, Inma
echo 'Hola, ' . $nombre;       // Hola, Inma
echo "Hola", " ", $nombre;     // Hola Inma

```

htmlspecialchars() : convierte ciertos caracteres especiales en su representación HTML segura.

```

$input = "<script>alert('Hola')</script>";
echo $input; //Salida peligrosa, se ejecuta el script

```

```

$input = "<script>alert('Hola')</script>";
echo htmlspecialchars($input); //Salida segura, se muestra en pantalla como texto,
no cómo código.

```

Existe una abreviatura cuando solo queremos mostrar el valor de una variable

```

<?php echo $variable; ?> // Esto sería el formato tradicional

```

```

<?= $variable ?> // El resultado es el mismo que el anterior, pero abreviado.

```

Sirve para imprimir directamente el contenido de una variable (u otra expresión) de forma más rápida y limpia.

Hay funciones que **muestran el contenido con más detalle** y son muy útiles en el desarrollo y para depuración

`print_r()`

```
$datos = [1, 2, 3];  
print_r($datos); // Salida: Array ( [0] => 1 [1] => 2 [2] => 3 )
```

- Útil para arrays y objetos
- No muestra tipos
- Se puede capturar con `print_r($var, true)`

`var_dump()`

```
$numero = 42;  
$texto = "hola";  
var_dump($numero); //Salida: int(42)  
var_dump($texto); //Salida: string(4) "hola"
```

- Muestra tipo y longitud

En PHP se pueden usar **comillas simples (')** o **dobles (")** para definir cadenas de texto, pero **no son equivalentes** y la elección influye en el comportamiento y el rendimiento.

Característica	Comillas simples '	Comillas dobles "
Interpolación de variables	No	Sí
Escape de caracteres especiales	Solo <code>\\</code> y <code>\'</code>	Muchos: <code>\n</code> , <code>\t</code> , <code>\"</code> , etc.
Rendimiento	Ligeramente más rápido	Ligeramente más lento

```
$nombre = "Inma";  
echo 'Hola $nombre\nHola otra vez';  
echo "Hola $nombre\nHola otra vez";  
  
//Salida  
// Hola $nombre\nHola otra vezHola Inma Hola otra vez
```

En realidad este ejemplo no es muy bueno, ya que el navegador no interpreta `\n` como un salto de línea, para ellos deberíamos poner `<br>` . Sin embargo, es para que veamos la diferencia entre comillas simples y comillas dobles. Esto nos puede llevar a plantearnos, ¿y si queremos escribir una comilla en HTML?, ¿cómo lo hacemos?

```
// Queremos escribir <input type="text" placeholder="Escribe tu nombre">

// Opción 1: Usar comillas simples en PHP para rodear el HTML (más limpia)
echo '<input type="text" placeholder="Escribe tu nombre">';

// Opción 2: Usar comillas dobles en PHP y escapar las del HTML. Funciona, pero menos elegante
echo "<input type=\"text\" placeholder=\"Escribe tu nombre\">";

// Opción no válida
echo "<input type="text" placeholder="Escribe tu nombre">";
```

9. Operadores lógicos, relacionales y de asignación

Los operadores **lógicos**, **relacionales** y **de asignación** son fundamentales en cualquier lenguaje de programación para realizar comparaciones, tomar decisiones y asignar valores a las variables. Tanto en **Java** como en **PHP**, estos operadores funcionan de manera similar, pero hay algunas diferencias en la flexibilidad y la tipificación. Vamos a repasarlos uno por uno, comparando ambos lenguajes.

9.1. Operadores Lógicos

Los operadores lógicos permiten combinar condiciones booleanas. Los más comunes son `AND` , `OR` y `NOT` .

Operadores Lógicos en Java y PHP:

Operador	Descripción	Java	PHP
<code>&&</code>	AND lógico	<code>cond1 && cond2</code>	<code>\$cond1 && \$cond2</code>
<code> </code>	OR lógico	<code>`cond1 </code>	<code>cond2`</code>
<code>!</code>	NOT lógico	<code>!cond</code>	<code>!\$cond</code>

Diferencias:

- Ambos lenguajes usan `&&` para **AND** y `||` para **OR**, pero PHP también tiene las versiones en palabras clave: `and` , `or` . Aunque PHP tiene el operador `and` y `or` , es mejor no usarlo porque su baja precedencia (menor precedencia que la asignación) puede causar errores sutiles en la evaluación de expresiones.
- En **Java**, los operadores lógicos funcionan solo con valores booleanos (`true` o `false`), mientras que en **PHP**, cualquier valor que sea considerado "falso" (como `0` , `null` , `''` , etc.) puede ser evaluado dentro de una condición lógica.

9.2. Operadores Relacionales

Los operadores relacionales son utilizados para comparar dos valores. Devuelven `true` o `false` dependiendo de si la comparación es verdadera o falsa.

Operadores Relacionales en Java y PHP:

Operador	Descripción	Java	PHP
<code>==</code>	Igual a	<code>a == b</code>	<code>\$a == \$b</code>
<code>!=</code>	No es igual a	<code>a != b</code>	<code>\$a != \$b</code>
<code>></code>	Mayor que	<code>a > b</code>	<code>\$a > \$b</code>
<code><</code>	Menor que	<code>a < b</code>	<code>\$a < \$b</code>
<code>>=</code>	Mayor o igual que	<code>a >= b</code>	<code>\$a >= \$b</code>
<code><=</code>	Menor o igual que	<code>a <= b</code>	<code>\$a <= \$b</code>
<code>===</code>	Igual a (incluyendo tipo)	No disponible	<code>\$a === \$b</code>
<code>!==</code>	No igual a (incluyendo tipo)	No disponible	<code>\$a !== \$b</code>

Diferencias:

- En **PHP**, además de los operadores estándar, existen `===` y `!==` , que comparan tanto el valor **como el tipo** de las variables. Esto no está disponible en **Java** porque Java es fuertemente tipado, lo que significa que las variables ya deben ser del mismo tipo para compararlas.

- **Ejemplo PHP:**

```

$a = 5;    // entero
$b = "5";  // cadena
var_dump($a == $b); // true, PHP convierte la cadena a entero
                // para comparar
var_dump($a === $b); // false, porque los tipos son diferentes
                // (entero vs cadena)

```

9.3. Operadores de Asignación

Los operadores de asignación permiten asignar valores a las variables y realizar operaciones combinadas.

Operadores de Asignación en Java y PHP:

Operador	Descripción	Java	PHP
=	Asignación simple	a = b	\$a = \$b
+=	Asignación con suma	a += b	\$a += \$b
-=	Asignación con resta	a -= b	\$a -= \$b
*=	Asignación con multiplicación	a *= b	\$a *= \$b
/=	Asignación con división	a /= b	\$a /= \$b
%=	Asignación con módulo	a %= b	\$a %= \$b

Diferencias:

- No hay muchas diferencias en cómo funcionan los operadores de asignación en ambos lenguajes. Tanto **Java** como **PHP** permiten usar operadores combinados como `+=`, `-=`, `*=`, y otros.
- **PHP** tiende a ser más permisivo con el tipo de datos en las asignaciones, mientras que en **Java**, como es fuertemente tipado, debes asegurarte de que las variables y operaciones sean del mismo tipo o realizar conversiones explícitas.

10. Arrays

Los **arrays** en PHP son estructuras de datos que permiten almacenar múltiples valores en una sola variable. Son muy flexibles y pueden contener cualquier tipo de dato, incluidos números, cadenas, otros arrays e incluso objetos. Si comparamos los arrays de Java frente a los de PHP

podemos ver que aunque el término es el mismo, **son estructuras muy distintas**. PHP tiene arrays muy flexibles y de alto nivel, mientras que Java es mucho más estricto.

Característica	PHP	Java
¿Tipo de array?	Array dinámico (vector, mapa, mixto)	Array fijo de tipo definido
¿Tamaño?	Dinámico	Fijo al declarar
¿Indexación?	Numérica o asociativa (claves strings o números)	Solo numérica (índices enteros)
Tipado	Débil (mezcla de tipos posible)	Fuerte (todos los elementos del mismo tipo)
Clases para arrays avanzados	No (usa arrays y objetos mezclados)	Sí: <code>ArrayList</code> , <code>HashMap</code> , etc.

Declaración de arrays

Java:

```
// Array de enteros
int[] numeros = {1, 2, 3};
// Array de cadenas
String[] nombres = {"Ana", "Luis"};
// No hay arrays "asociativos" → se usan mapas
Map<String, Object> persona = new HashMap<>();
persona.put("nombre", "Inma");
persona.put("edad", 30);
```

PHP:

```
// Indexado
$numeros = [1, 2, 3];
// Asociativo
$persona = ["nombre" => "Inma", "edad" => 30];
// Mixto (¡válido!)
$mixto = [1, "nombre" => "Inma", 2];`
```

Acceso a elementos

Lenguaje	Acceso numérico	Acceso asociativo
PHP	<code>\$numeros[0]</code>	<code>\$persona["nombre"]</code>
Java	<code>numeros[0]</code>	<code>persona.get("nombre")</code>

Recorrer arrays

Java:

```
for (Map.Entry<String, Object> entry : persona.entrySet()) {  
    System.out.println(entry.getKey() + ": " + entry.getValue());  
}
```

PHP:

```
foreach ($persona as $clave => $valor) {  
    echo "$clave: $valor\n";  
}
```

Tipos de arrays en PHP

10.1. Arrays indexados (numéricos):

Estos arrays utilizan **números enteros** como índice. Los índices comienzan desde 0 de manera predeterminada.

Ejemplo:

```
$frutas = array("Manzana", "Naranja", "Banana");  
echo $frutas[0];  
// Imprime "Manzana"
```

A partir de PHP 5.4, también puedes declarar un array usando la sintaxis de corchetes:

```
$frutas = ["Manzana", "Naranja", "Banana"];  
echo $frutas[1];  
// Imprime "Naranja"
```

Aquí, `Manzana` está en el índice `0`, `Naranja` en el índice `1`, y `Banana` en el índice `2`.

10.2. Arrays asociativos:

En los arrays asociativos, puedes usar **cadenas** como claves (índices) en lugar de números enteros. Esto es útil cuando quieres asociar valores con nombres o etiquetas específicas.

Ejemplo:

```
$persona = array("nombre" => "Juan", "edad" => 25, "ciudad" => "Madrid" );  
echo $persona["nombre"];  
// Imprime "Juan"
```

En este caso, en lugar de acceder a los valores usando índices numéricos, se utilizan las claves "nombre", "edad" y "ciudad". A los **arrays asociativos** se puede acceder utilizando **comillas dentro de los corchetes** o sin utilizarlas. No utilizarlas funciona, aunque es incorrecto (o más bien se desaconseja totalmente su uso).

10.3. Arrays multidimensionales:

Un array multidimensional es un array que contiene otros arrays. Son útiles cuando necesitas representar estructuras más complejas, como tablas o matrices.

Ejemplo:

```
$matriz = array(array(1, 2, 3), array(4, 5, 6), array(7, 8, 9) );  
echo $matriz[1][2];  
// Imprime "6"
```

En este caso, `$matriz[1]` hace referencia al segundo array (con valores 4, 5, 6), y `$matriz[1][2]` accede al tercer valor de ese array, que es 6.

El array mutlidimensional es muy cómo si por ejemplos queremos almacenar dentro de una misma tabla el nombre, moneda y lengua hablada en cada país. Para hacerlo podemos emplear un array llamado país que vendrá definido por estas tres características (claves). Para crearlo, deberíamos escribir una expresión del mismo tipo que la vista anteriormente en la que meteremos una array dentro del otro. Este proceso de incluir una instruccion dentro de otra se llama anidar y es muy corriente en programación:

```
<?
$país=array
(
    "espana" =>array
        (
            "nombre"=>"España",
            "lengua"=>"Castellano",
            "moneda"=>"Peseta"
        ),
    "francia" =>array
        (
            "nombre"=>"Francia",
            "lengua"=>"Francés",
            "moneda"=>"Franco"
        )
);
echo $país["espana"]["moneda"]; //Saca en pantalla: "Peseta"
```

Otro ejemplo, podemos combinar un array indexado y un array asociativo

```

<?php
$personas = [
    [
        "nombre" => "Inma",
        "edad" => 35,
        "lengua" => "Castellano"
    ],
    [
        "nombre" => "Jean",
        "edad" => 28,
        "lengua" => "Francés"
    ],
    [
        "nombre" => "Anna",
        "edad" => 40,
        "lengua" => "Inglés"
    ]
];

// Mostrar el nombre de la primera persona
echo $personas[0]["nombre"]; // Salida: Inma

// Recorrer todo el array
foreach ($personas as $persona) {
    echo $persona["nombre"] . " habla " . $persona["lengua"] . "<br>";
}

```

10.4. Funciones comunes para trabajar con arrays

PHP tiene muchas funciones útiles para manipular arrays:

Agregar elementos a un array:

Se puede agregar un elemento a un array con la función `array_push()` o con la sintaxis de corchetes `[]`.

```

$frutas = ["Manzana", "Naranja"];
array_push($frutas, "Banana");
$frutas[] = "Fresa";
// Otra forma de agregar un elemento

$personas = [
    "nombre" => "Inma",
    "edad" => 35,
    "lengua" => "Castellano"
];

$personas["pais"] = "Sevilla";

print_r($frutas);
// Imprime Array ( [0] => Manzana [1] => Naranja [2] => Banana [3] => Fresa )

```

Eliminar el último elemento de un array:

Con la función `array_pop()` puedes eliminar el último elemento de un array.

```

$frutas = ["Manzana", "Naranja", "Banana"];
array_pop($frutas);

print_r($frutas);
// Imprime Array ( [0] => Manzana [1] => Naranja )

```

Obtener elementos de un array

Para disminuir el número de casillas de un arreglo tenemos varias funciones. Entre ellas, `array_slice()` la utilizamos cuando queremos obtener algunos elementos del array, sabiendo los índices de los elementos que deseamos obtener.

Recibe tres parámetros. El array, el índice del primer elemento y el número de elementos a tomar, siendo este último parámetro opcional.

```
<?php
```

```
$entrada = array ("Miguel", "Pepe", "Juan", "Julio", "Pablo");
```

```
// Obtengo un array con los tres primeros elementos
```

```
$salida = array_slice ($entrada, 0, 3);
```

```
// Array ( [0] => Miguel [1] => Pepe [2] => Juan )
```

```
// Obtengo el array desde el primer elemento hasta el final
```

```
$salida = array_slice ($salida, 1);
```

```
// Array ( [0] => Pepe [1] => Juan )
```

"Sacar" el primer elemento del array:

`arrayshift()` extrae el primer elemento del array y lo devuelve. Además, acorta la longitud del array eliminando el elemento que estaba en la primera casilla. Siempre hace lo mismo, por tanto, no recibirá más que el array al que se desea eliminar la primera posición.

```
<?php
```

```
entrada = array ("Miguel", "Pepe", "Juan", "Julio", "Pablo");
```

```
//quito la primera casilla
```

```
$salida = array_shift ($entrada);
```

```
//muestro el array
```

```
echo "La función devuelve: " . $salida . "<br>"; // La función devuelve: Miguel
```

```
print_r($entrada);
```

```
// Array ( [0] => Pepe [1] => Juan [2] => Julio [3] => Pablo )
```

```
//quito la primera casilla, que ahora sería la segunda del array original
```

```
$salida = array_shift ($entrada);
```

```
echo "La función devuelve: " . $salida . "<br>"; // La función devuelve: Pepe
```

Recorrer un array:

Para iterar sobre los valores de un array, puedes usar un bucle `foreach`.

```

$frutas = ["Manzana", "Naranja", "Banana"];
foreach ($frutas as $fruta) {
    echo $fruta . "<br>";
}
// Imprime:
// Manzana
// Naranja
// Banana

```

Si necesitamos tanto las claves como los valores de un array asociativo, podemos usar esta sintaxis:

```

$persona = ["nombre" => "Juan",
            "edad" => 25,
            "ciudad" => "Madrid" ];
foreach ($persona as $clave => $valor) {
    echo "$clave: $valor<br>";
}

// Imprime:
// nombre: Juan
// edad: 25
// ciudad: Madrid

```

Buscar un elemento en un array

Para buscar un elemento en un array podemos usar dos funciones, `in_array()` y `array_search()`.

La función `in_array()` busca un valor en un array y devuelve `true` si lo encuentra y `false` en otro caso. Recibe tres parámetros, el valor buscado, el array y un tercer parámetro opcional que indica si la búsqueda es estricta o no.

```

<?php
$colores = ["rojo", "verde", "azul"];

if (in_array("verde", $colores)) {
    echo "¡Verde está en el array!";
} else {
    echo "No está.";
}

```

La función `array_search()` devuelve la **clave o índice** donde se encuentra el valor. Si no lo encuentra, devuelve `false`. Admite modo estricto también.

```

<?php
$frutas = ["manzana", "pera", "naranja"];

$pos = array_search("naranja", $frutas);
if ($pos !== false) {
    echo "La fruta está en la posición $pos";
} else {
    echo "No se encontró";
}

$usuarios = [
    "user1" => "Inma",
    "user2" => "Luis",
    "user3" => "Ana"
];

$buscado = "Luis";
$clave = array_search($buscado, $usuarios);

if ($clave !== false) {
    echo "El usuario '$buscado' tiene la clave '$clave'.";
} else {
    echo "Usuario no encontrado.";
}

```

Contar elementos en un array:

La función `count()` te permite obtener el número de elementos en un array.

```

$frutas = ["Manzana", "Naranja", "Banana"];
echo count($frutas);

// Imprime "3"

```

"Destruir el array"

`unset()` se utiliza para destruir una variable dada. En el caso de los arrays, se puede utilizar para eliminar un array completo o una casilla de un array.

```
<?php
```

```
$tabla = array ("Lagartija", "Araña", "Perro", "Gato", "Ratón");
```

```
$tabla2 = array (12,34,45,52,12);
```

```
$tabla3 = array ("Sauce", "Pino", "Naranja", "Chopo", "Perro", "34");
```

```
//aumentamos el tamaño del array
```

```
$resultado = array_merge($tabla, $tabla2, $tabla3);
```

```
print_r($resultado);print_r($estadios_futbol);
```

```
// Array ( [Barcelona] => Nou Camp [Real Madrid] => Santiago Bernabeu [Valencia] => Mestalla [Real Sociedad] => Anoeta )
```

```
//eliminamos el estadio asociado al real madrid
```

```
unset ($estadios_futbol["Real Madrid"]);
```

```
print_r($estadios_futbol);
```

```
// Array ( [Barcelona] => Nou Camp [Valencia] => Mestalla [Real Sociedad] => Anoeta )
```

Unir varios arrays:

La función `array_merge()` se utiliza para unir dos o más array en uno solo. Se le pasan dos o más arrays por parámetro y devuelve un array con todos los campos de los arrays pasados.

```
<?php
```

```
$tabla = array ("Lagartija", "Araña", "Perro", "Gato", "Ratón");
```

```
$tabla2 = array ("12", "34", "45", "52", "12");
```

```
$tabla3 = array ("Sauce", "Pino", "Naranja", "Chopo", "Perro", "34");
```

```
//aumentamos el tamaño del array
```

```
$resultado = array_merge($tabla, $tabla2, $tabla3);
```

```
print_r($resultado);
```

```
// Array ( [0] => Lagartija [1] => Araña [2] => Perro [3] => Gato [4] => Ratón [5] => 12 [6] => 34 [7] => 45 [8] => 52 [9] => 12 [10] => Sauce [11] => Pino [12] => Naranja [13] => Chopo [14] => Perro [15] => 34 )
```

Ordenar un array:

PHP tiene funciones como `sort()` para ordenar arrays en orden ascendente.

```
$numeros = [4, 2, 8, 1];  
sort($numeros);  
print_r($numeros);  
  
// Imprime Array ( [0] => 1 [1] => 2 [2] => 4 [3] => 8 )`
```

Para arrays asociativos, podemos usar `asort()` para ordenar los valores manteniendo las claves.

```
$persona = ["nombre" => "Carlos", "edad" => 30, "ciudad" => "Barcelona" ];  
  
asort($persona);  
  
print_r($persona);  
  
// Ordena el array por los valores, pero mantiene las claves`
```

Funciones básicas de cadena

Función	Descripción	Ejemplo	Resultado
<code>strlen()</code>	Devuelve la longitud de una cadena.	<code>strlen("Hola")</code>	4
<code>strtoupper()</code>	Convierte una cadena a mayúsculas.	<code>strtoupper("hola")</code>	"HOLA"
<code>strtolower()</code>	Convierte una cadena a minúsculas.	<code>strtolower("HOLA")</code>	"hola"
<code>ucfirst()</code>	Convierte en mayúscula la primera letra.	<code>ucfirst("hola mundo")</code>	"Hola mundo"
<code>ucwords()</code>	Mayúsculas al inicio de cada palabra.	<code>ucwords("hola mundo")</code>	"Hola Mundo"
<code>trim()</code>	Elimina espacios al inicio y fin.	<code>trim(" hola ")</code>	"hola"
<code>substr()</code>	Extrae parte de una cadena.	<code>substr("Hola mundo", 5, 5)</code>	"mundo"
<code>strpos()</code>	Posición de la primera ocurrencia.	<code>strpos("Hola mundo", "mundo")</code>	5
<code>str_replace()</code>	Reemplaza texto.	<code>str_replace("mundo", "PHP", "Hola mundo")</code>	"Hola PHP"
<code>explode()</code>	Divide una cadena por un delimitador.	<code>explode(",", "a,b,c")</code>	["a", "b", "c"]

Función	Descripción	Ejemplo	Resultado
<code>preg_split()</code>	Divide una cadena por una expresión regular .	<code>preg_split("/[\s,]+/", "a, b c")</code>	<code>["a", "b", "c"]</code>
<code>implode()</code> / <code>join()</code>	Une elementos de un array en una cadena.	<code>implode("-", ["2025", "07", "27"])</code>	<code>"2025-07-27"</code>
<code>htmlspecialchars()</code>	Escapa caracteres HTML (útil en formularios).	<code>htmlspecialchars("Hola")</code>	<code>"&lt;b&gt;Hola&lt;"</code>
<code>nl2br()</code>	Convierte saltos de línea (<code>\n</code>) en <code>
</code> .	<code>nl2br("Hola\nMundo")</code>	<code>"Hola
Mundo"</code>
<code>strcmp()</code>	Compara cadenas (sensible a mayúsculas).	<code>strcmp("hola", "Hola")</code>	<code>≠ 0</code>
<code>strrev()</code>	Invierte una cadena.	<code>strrev("Hola")</code>	<code>`"aloH"</code>
<code>str_split()</code>	convierte una cadena en un array.		

Las funciones `mb_*` son preferibles si se trabaja con UTF-8 y tildes (ej: `mb_strlen()`).